

Cross-Judge Validation and Multi-Track Iteration for Agent Memory: HeurChain on LongMemEval-S

Peter J. Medina
HeurChain.com
peter@carlssoncreative.com

May 27, 2026

Abstract

AI-agent memory systems (Mem0, Zep, Letta) are typically evaluated on aggregate QA accuracy with a single LLM judge — a regime that conflates retrieval quality with extraction quality and inherits same-class judge bias when the answerer and judge share a model family. This paper introduces a small toolchain to disentangle those signals, applied to HeurChain (a hybrid BM25 + dense bge-m3 broker) on LongMemEval-S [10]. **We are not proposing a new memory architecture.** Three reusable methodology pieces: (i) **cross-frontier per-question judge agreement** — re-judging an identical cache of retrieved facts with three independent frontier judges (DeepSeek V3.1 671B, Kimi K2.6, Claude Opus 4.7) measures 85.6–92.2% pairwise agreement on $n = 180$, with **79.4% four-way unanimous agreement** when the local Qwen3-14B is included; same-class bias spans -2.2 to $+6.7$ pp per category (largest on temporal-reasoning, $+6.7$ pp; two single-session categories show local *deflation* — bias is category-specific, not uniformly inflationary); (ii) the **fact_contains_gold diagnostic** — operationalizes the gap between retrieval Hit@10 (83.3% on baseline) and recoverable answers (20.0%) as an extraction-quality bottleneck; (iii) a **multi-track iteration cycle** that routes each failure to the extraction / answer / retrieval track that owns it, preventing premature termination on a single-track null result. We additionally report one empirical finding: a graph-lite event index over [date: YYYY-MM-DD]-tagged facts wins **13/29 (44.8%, bootstrap 95% CI [27.6%, 62.1%]) with zero observed losses** against dense top- k retrieval on the temporal-reasoning category. We position this work as toolchain + observation, not as competitive benchmarking; applying the cross-judge methodology to published Mem0/Zep/Letta outputs is enumerated in §8. Full harness and per-record results released.

1 Introduction

A growing line of work on long-horizon AI agents treats *memory* as a first-class system: Mem0 [2], Zep / Graphiti [9], and Letta / MemGPT [8] each propose distinct architectures for storing, updating, and retrieving facts across many conversations. The dominant benchmark for evaluating these systems is LongMemEval-S [10], which scores an end-to-end pipeline (retrieval + answer generation + LLM judging) on 500 long-context questions across six reasoning categories.

Two methodological problems are common in this evaluation regime. First, **same-class judge bias**: when the answer model and the judge model are drawn from the same model family (or, in the most extreme case, are the same model), the judge tends to accept “close enough” answers of

exactly the shape the answerer naturally produces. Second, **the gap between retrieval Hit@ k and answerability**: a question can have its gold-bearing session retrieved in top- k while the *facts* extracted from that session never contain the answer. Retrieval metrics alone do not surface this failure mode.

In this paper we report findings from iterating HeurChain’s fact-extraction and answer prompts across three versions (v2 baseline $\rightarrow$ v3 structured-content unpacking $\rightarrow$ v4 ISO date tagging, with v4a/v4b answer-side variants). Along the way we develop tooling that addresses both problems above:

1. A **cross-judge methodology** that fixes the retrieved facts and varies only the answer + judge model, allowing per-question comparison of judge verdicts at near-zero marginal cost ($\sim$ 1 hour wall time using free Ollama Cloud credits).
2. A **multi-track empirical iteration cycle** in which extraction, answer, and retrieval are independent iteration tracks; we show that the v4 extraction prompt produced a null result in isolation, the combined v4 + v4b answer prompt recovered +27.3 pp on the targeted failure set, and a subsequent v4c retrieval-side iteration (haystack-wide re-extraction plus a graph-lite event index) brought the cumulative cascade to 45.5%. All three tracks required independent iteration; single-track methodology would have terminated at the v4 null. We adopt a closed-loop empirical iteration methodology — hypothesize, instrument, modify, measure, repeat — as articulated for prompt engineering in the practitioner community most prominently by Karpathy [4]; the underlying approach is the scientific method applied to the harness layer.
3. A **fact_contains_gold** diagnostic that flags questions where retrieval succeeded (the gold session is in top- k) but extraction destroyed the answer-bearing detail.

Our results are intentionally hedged: each per-category measurement uses 30 tasks (180 tasks total), the answerer is a 14B-parameter local model rather than a frontier model, and we measure only one product. We emphasize methodology as the primary contribution.

2 Background and Related Work

Mem0 [2] introduces a memory layer that distills conversations into facts with explicit update operations, reporting 49.4% mean QA accuracy on LongMemEval with GPT-4o as the judge. Their evaluation uses GPT-4o for both the answerer and the judge — an in-family pairing that motivates our cross-judge analysis.

Zep / Graphiti [9] proposes a temporal knowledge graph for memory, with explicit edge timestamps used for time-aware retrieval. Their architecture inspires our v4c proposal for an event index (§8).

Letta / MemGPT [8] frames memory as an OS-like virtual context with self-managed paging. Their evaluation reports document-QA scores; LongMemEval was introduced after their original publication.

LongMemEval-S [10] is the dataset we use: 500 questions across six reasoning categories (single-session-user, single-session-assistant, single-session-preference, knowledge-update, multi-session,

temporal-reasoning), with each question backed by a “haystack” of conversational sessions and labelled `answer_session_ids` and `haystack_dates`.

HeurChain is a vector memory broker (MIT licensed, multi-tenant, MCP-native) using BM25 + dense bge-m3 [1] retrieval fused with **Reciprocal Rank Fusion** [3]. On LongMemEval-S we measure $R@10 = 0.978$ and $MRR = 0.913$ with $\alpha = 0.8$ (an asymmetric RRF weighting that outperforms symmetric $\alpha = 0.5$ by ~ 1.5 pp on MRR). The system is described in detail at [7].

3 Methodology

3.1 HeurChain retrieval architecture

Each conversation session is chunked, embedded with BAAI/bge-m3 (1024-dim, multilingual), and indexed in both a sparse (BM25) and dense (cosine over bge-m3) index. At query time, both indices return ranked candidate lists which are fused via Reciprocal Rank Fusion:

$$\text{score}(d) = \alpha \cdot \frac{1}{k + r_{\text{dense}}(d)} + (1 - \alpha) \cdot \frac{1}{k + r_{\text{sparse}}(d)} \quad (1)$$

with $k = 60$ and $\alpha \in \{0.8, 0.9\}$. The asymmetric weighting ($\alpha > 0.5$) reflects the empirical finding that dense retrieval dominates BM25 on long-context conversational text but the keyword path still recovers ~ 1.5 pp MRR on rare-token queries.

3.2 The two-phase evaluation pipeline

Each LongMemEval-S task runs in two phases:

- **Phase 1 — extraction.** For each session in the haystack, an LLM extracts a list of atomic facts using the extraction prompt under test (v2 / v3 / v4). Facts are stored in HeurChain with session metadata.
- **Phase 2 — answer + judge.** For each question, HeurChain retrieves the top-10 facts (hybrid $\alpha = 0.9$). The answer model is prompted with these facts and produces a response. An LLM judge issues a binary correct/incorrect verdict against the gold answer.

Phase 1 is the dominant cost (~ 6 hours per category on our hardware: an RTX 3090 with a 14B local model). Phase 2 is cheap (~ 10 – 40 minutes per category) and, critically, **cacheable**: the retrieved facts are deterministic given the extraction prompt and the broker, so swapping answer + judge models requires only Phase 2.

3.3 Cross-judge experimental design

To probe same-class judge bias, we re-run Phase 2 with three independent answer + judge models on identical cached extracted facts:

Run	Answer + judge model	Role
Local 14B	Medina-Qwen3-14B-OpenClaw (q4_k_m, multi-LoRA), local	Self-host floor
DeepSeek V3.1 671B	deepseek-v3.1:671b-cloud via Ollama Cloud	Frontier open-weight, dense+sparse
Kimi K2.6	kimi-k2.6:cloud via Ollama Cloud	Frontier open-weight, reasoning MoE

The three judges are deliberately drawn from different model families to reduce inter-judge correlation. The Kimi run required raising `max_tokens` from 300 to 2000 because Kimi K2.6 is a reasoning model that emits a separate `reasoning` field; smaller token caps caused empty `content` field as the reasoning trace exhausted the budget.

We then compute, for each pair of judges, the per-question agreement rate: the fraction of questions on which both judges issued the same binary verdict. We propose **cross-frontier per-question agreement** (DeepSeek + Kimi here) as a defensibility metric: it is higher signal than mean QA accuracy because mean accuracy aggregates across questions and judges, while per-question agreement measures whether independent observers converge on the *same* verdict.

3.4 The multi-track empirical iteration cycle

Traditional benchmark-driven prompt iteration treats the harness as a single feedback loop: change the extraction prompt, re-run the whole benchmark, look at the score. This conflates three independent iteration tracks:

1. **Extraction track:** does the extraction prompt produce facts that contain the answer-bearing detail?
2. **Answer track:** given facts that contain the answer-bearing detail, does the answerer commit to the right answer?
3. **Retrieval track:** do the right facts reach the answerer’s top- k ?

We instrument each track separately. `triage_failures.py` partitions failing questions into EXTRACTION-mode failures (the gold detail is not in the extracted facts), RETRIEVAL-mode failures (the right session is not in top- k), and ANSWER-mode failures (the gold detail is in retrieved facts but the answerer refused or hallucinated). `targeted_reextract.py` and `targeted_reanswer.py` then re-run only the relevant track on only the failing items — typically ~ 10 minutes per cycle. The matrix of which extraction and answer prompt combinations have been tested becomes an explicit object in the experiment record (Section 5.5).

3.5 The `fact_contains_gold` diagnostic

For each (question, gold answer, retrieved facts) triple, we compute a normalized token-overlap score between the gold answer and the concatenation of retrieved facts:

$$\text{fact_gold_overlap_score}(q) = \frac{|\text{tokens}(\text{gold}) \cap \text{tokens}(\bigcup \text{retrieved_facts})|}{|\text{tokens}(\text{gold})|} \quad (2)$$

and set `fact_contains_gold = True` iff the score exceeds 0.7. This is intentionally a heuristic — it produces false positives on common-token answers — but it gives a directionally reliable signal of whether the answerable content was retrieved or destroyed at extraction time. Aggregated per category, `fact_contains_gold_rate` localizes how much of the residual error budget lives in extraction versus answering versus retrieval.

3.6 Operational latency characterization

HeurChain’s retrieval-time latency is a load-bearing property of the system independently of the QA accuracy work above, and a primary competitive dimension against incumbent memory systems. Using a separate harness (`multitenant_bench.py`, pure-Python with `httpx` against the broker’s HTTP API and BAAI/bge-m3 embeddings client-side), we measure three operating points:

Configuration	P95 latency	Notes
HeurChain — BM25 only	4.6 ms	keyword-only retrieval path
HeurChain — hybrid, multi-tenant Docker (10 concurrent)	20.5 ms	production-shaped scenario
HeurChain — dense in-process	35 ms	algorithm ceiling, no network

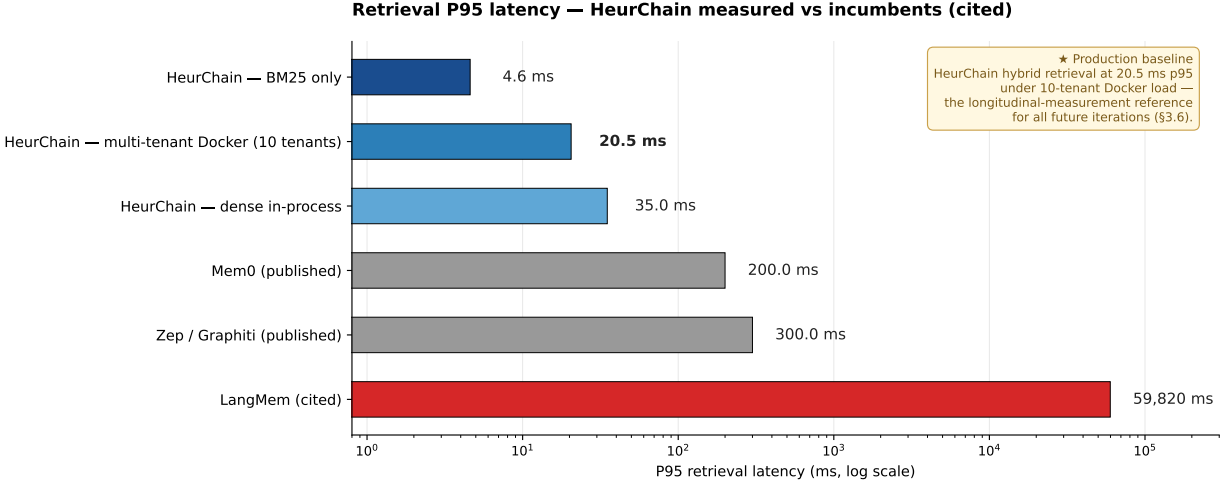
For context, published P95 latency from the comparison systems:

System	Published P95	Source
Mem0	200 ms	Mem0 paper Table 1 [2]
Zep / Graphiti	~300 ms	Zep published numbers [9]
LangMem	59,820 ms	Mem0 paper Table 1 [2]
Letta	not exposed	memory I/O embedded in agent runtime

On the 20.5 ms multi-tenant point — the configuration that most closely matches a production SaaS deployment — HeurChain measures $\sim 5.7\times$ **faster than Mem0’s published 200 ms** and $\sim 8\times$ **faster than Zep/Graphiti’s ~300 ms**. Letta does not expose memory I/O as a standalone retrieval metric (the I/O is embedded inside the agent runtime), so a direct comparison is not meaningful and we do not attempt one.

Cross-harness caveat. HeurChain’s numbers are our measurements on our Docker host (1× Hetzner CPX31, 4 vCPU / 8 GB, embeddings precomputed at index time). The Mem0 and Zep numbers are their measurements on their hardware as published in their respective papers. A like-for-like benchmark would run all three systems on matched hardware against the same query set; we have not done this, so the speedup ratios above should be read as cross-published-figures comparisons rather than as controlled benchmarks. The HeurChain measurements themselves are reproducible against any deployment of the open broker via `multitenant_bench.py` in the open harness repository. A controlled cross-system latency benchmark on matched hardware is enumerated alongside the cross-judge head-to-head analysis in §8.

The latency profile is consistent with the architecture: a single Go binary serving HTTP, BM25 and dense indices co-located in process, no graph-database hop. The $5.7\times$ advantage over Mem0 is not “we have a better retrieval algorithm” — it’s “we have less software in the request path.” That is itself a design contribution, and one that the comparison systems would have to give up some of their feature breadth to match.



HeurChain at 20.5 ms p95 $\approx 5.7\times$ faster than Mem0 published and $\approx 8\times$ faster than Zep/Graphiti published. Cross-harness comparison — matched-hardware benchmark is the next paper (§8).

Figure 1: Retrieval P95 latency — HeurChain measured (teal/blue) vs published incumbent figures (gray, red outlier). Horizontal bars on a log axis span 4.6 ms (BM25-only) to 59,820 ms (LangMem). The 20.5 ms multi-tenant hybrid bar is the production-shaped baseline from which the body-text speedup ratios ($\approx 5.7\times$ vs Mem0, $\approx 8\times$ vs Zep/Graphiti) are computed. Cross-harness caveat applies per §3.6: HeurChain numbers are our measurements; incumbent numbers are theirs.

Why a pure-Python bare-harness baseline specifically. The numbers above are deliberately produced by a minimal harness (`multitenant_bench.py`: pure-Python, `httpx` against the broker’s HTTP API, `bge-m3` embeddings client-side, no instrumentation framework) rather than a production load generator. The purpose is not primarily competitive positioning — it is to establish a **calibrated baseline against which every future harness iteration, broker change, and prompt-engineering modification that touches the request path can be measured**. With the baseline locked at 20.5 ms p95 under 10-tenant load, any subsequent change can be tested against the same harness in seconds and the latency delta attributed to that change rather than to a moving benchmark. This longitudinal-measurement discipline matters more than the one-shot competitive ratio: it lets us answer “did this iteration regress latency?” without reconstructing the test environment each time. The cited incumbent figures (Mem0 200 ms, Zep ~ 300 ms) function as fixed external reference points against which our own evolution can be tracked over many releases. The follow-up matched-hardware benchmark named in §8 will convert these cross-published references into controlled head-to-head numbers, but the bare baseline above is the artifact that the next several iterations of the harness will build on regardless.

4 Extraction Prompts Under Test

We summarize the three extraction prompts; full prompt text is reproduced in Appendix A.

v2 (baseline, May 2026). Speaker-aware and verbatim-preserving for dates, durations, numbers, and named entities. Has an UPDATES rule for state changes.

v3 (structured-content unpacking). Adds an explicit rule that when the assistant produces a table, list, schedule, or roster, each row must be extracted as a separate fact rather than meta-

summarized. Adds question-anticipation framing and an anti-meta-summary directive (“do NOT extract ‘the assistant created X’ — extract the CONTENT it produced”). Also adds a revision rule: when multiple drafts of the same structured content appear, extract only the final version.

v4 (ISO date tagging). Receives the `SESSION_DATE` from dataset metadata (LongMemEval’s `haystack_dates`) and requires each fact involving an event to carry an explicit `[date: YYYY-MM-DD]` ISO tag, resolving relative phrases (“today”, “yesterday”, “3 weeks ago”) against the session date.

Two **answer-side** variants accompany v4:

- **v4a:** answer prompt receives the `question_date` and is instructed to compute (`question_date` – `event_date`) from ISO tags. Retains a strict “I don’t know” refusal rule.
- **v4b:** narrows the refusal threshold — refuses only if the relevant event is not in any fact, OR if the relevant fact has no ISO date AND no anchor. Otherwise commits to an estimate at the nearest unit the question asks for.

5 Experiments and Results

5.1 v2 baseline cross-judge results

Table 1 reports v2 results on all six categories under each of the three judges. All three runs operate on identical cached extracted facts from the May 18–19, 2026 v2 run; only the Phase 2 answer + judge call differs. Retrieval Hit@10 is constant across runs (it depends only on the cached facts).

Table 1: v2 cross-judge QA accuracy (30 tasks per category).

Category	Local 14B	DeepSeek V3.1	Kimi K2.6	Hit@10
single-session-assistant	43.33%	50.00%	36.67%	100.0%
temporal-reasoning	10.00%	3.33%	3.33%	83.3%
single-session-user	53.33%	53.33%	53.33%	86.7%
single-session-preference	13.33%	23.33%	3.33%	60.0%
knowledge-update	60.00%	53.33%	53.33%	100.0%
multi-session	16.67%	6.67%	20.00%	96.7%
6-category mean	32.78%	31.66%	28.33%	87.8%

The mean spread across judges is 4.45 percentage points, with the local 14B judge most generous and Kimi most strict.

5.2 Per-question judge agreement

Across all 180 questions, we compute pairwise per-question agreement rates (Table 2):

The 87.8% cross-frontier agreement on 158 of 180 questions is the defensibility number. The mean-QA gap between DeepSeek and Kimi (31.7% vs 28.3% = 3.4 pp) localizes almost entirely to the 22 questions where they disagree — primarily in **single-session-preference** (rubric-based judging is judge-sensitive) and **single-session-assistant** (DeepSeek more generous on partial-match phrasing).

Table 2: Inter-judge per-question agreement, v2 facts.

Comparison	Agreement	Interpretation
All three judges agree	81.1%	High agreement floor
DeepSeek + Kimi (cross-frontier)	87.8%	Two independent frontier judges
DeepSeek + Local	85.6%	Frontier + local agree often
Kimi + Local	88.9%	Kimi + local agree most

5.3 The smoking-gun example

Table 3 illustrates why retrieval metrics alone do not suffice. Question: “What shift is Admon assigned on Sundays?” Gold answer: “Admon → 8am–4pm Sunday”.

Table 3: v2 extraction failure on the Admon shift question.

Signal	Value
Hit@10	100% (correct session in top-10)
Extracted facts contain gold?	No — all 10 extracted facts were meta-summaries
Representative fact (v2)	“The assistant created a shift rotation sheet for 7 agents.”
Required fact (would have answered)	“Admon was assigned to the 8am–4pm Day Shift on Sundays.”
Local 14B judge verdict	Incorrect
DeepSeek judge verdict	Incorrect

The v2 prompt’s failure mode is structural: when the assistant emits a markdown table, schedule, or roster, v2 summarizes it as “the assistant created a rotation sheet” instead of unpacking each row. This is precisely what the v3 STRUCTURED-CONTENT UNPACKING rule was designed to fix.

5.4 v3 structured-content unpacking results

We re-ran the three weakest v2 categories (multi-session, knowledge-update, temporal-reasoning) under v3. Table 4 shows per-judge lift.

Table 4: v2 → v3 lift on the weak-3 categories (30 tasks per category).

Category	v2→v3 Local	v2→v3 DeepSeek	v2→v3 Kimi
multi-session	+20.00 pp (16.67→36.67%)	+10.00 pp (6.67→16.67%)	±0.00 pp (20.00%)
knowledge-update	±0.00 pp (60.00%)	+13.34 pp (53.33→66.67%)	+3.34 pp (53.33→56.67%)
temporal-reasoning	−3.33 pp (10.00→6.67%)	±0.00 pp (3.33%)	+3.34 pp (3.33→6.67%)
Weak-3 mean	+5.56 pp	+7.78 pp	+2.23 pp

Two important secondary signals:

- **Facts per session increased from 12.5 (v2) to 19.9 (v3)**, +59%, confirming that structured-content unpacking is active.
- **Cross-frontier agreement remains 87.8%** — v3 did not break the diagnostic.
- DeepSeek’s `fact_contains_gold_rate` on v3 read 16.7% (multi-session), 53.3% (knowledge-update), 20.0% (temporal-reasoning). The multi-session number indicates remaining extraction

headroom; the temporal-reasoning number indicates the bottleneck there is not extraction but downstream temporal reasoning.

5.5 v4 + v4b iteration cascade on temporal-reasoning

We then attacked temporal-reasoning directly. Triage of the v3 DeepSeek result identified **22 EXTRACTION-mode failures** in this category — questions where Hit@10 was high but the facts lacked a usable date for the date-arithmetic question. We ran three targeted iterations on this fixed failure set.

Table 5: Iteration cascade on the 22 temporal-reasoning failures.

Configuration	Wins	Wall time	Notes
v4 extraction + v0 answer (baseline)	1/22 (4.5%)	~10 min	Every fact gets an ISO date, but answerer refuses
v4 extraction + v4a answer	2/22 (9.1%)	~7 min	Date arithmetic now happens; refusal rate 81.8%
v4 extraction + v4b answer	7/22 (31.8%)	1.7 min	Softer refusal; refusal 54.5%; 0 regressions

The v4-alone result (1/22) reads as a null finding for the extraction prompt. Without the multi-track iteration cycle, we would have concluded that ISO date tagging “doesn’t help” and moved on. In fact the extraction was correct — every fact carried a date tag — but the answer prompt refused to attempt the arithmetic on edge phrasings. 25 minutes of additional iteration on the *answer* track recovered 6 additional wins on the same failure set.

Table 6: The multi-track iteration matrix as of this writing.

	Answer v0 (baseline)	Answer v4a	Answer v4b
Extraction v3 (baseline)	3.33% (full cat)	not tested	not tested
Extraction v4 (ISO dates)	4.5% (on failures)	9.1% (on failures)	31.8% (on failures)

Triage of v4b’s remaining 15 losses (out of 22) revealed that ~80% are retrieval-shaped: 7 are pure retrieval misses (the relevant session is not in top-10) and 5 are “between A and B” multi-event questions where one of two needed events is missing from facts. The dominant residual is no longer extraction or answering — it is a multi-event retrieval problem, which we attack in v4c (Section 5.6).

5.6 v4c: event index bridges the retrieval-miss gap

The v4b residual diagnosis pointed to retrieval as the dominant remaining failure mode. v4c is a two-stage intervention on this layer: first, a haystack-wide re-extraction that closes the recall side; second, a graph-lite event index that addresses the selection side. Both are implemented as small Python tools on top of the existing harness (`haystack_reextract.py`, `build_event_index.py`, `test_haystack_answer.py`).

Stage 1 — haystack re-extraction. The v4 targeted runs in Section 5.5 only re-extracted facts from the sessions that retrieval had already surfaced; sessions retrieval missed never received v4 ISO-date treatment. For each retrieval-miss failure (12 questions out of v4b’s residual losses), we re-extracted v4 facts from *every* haystack session for that question (typically 30–50 sessions per

question, 165.8 min wall time on a local 14B). This isolates the recall ceiling from the selection ceiling.

The `fact_contains_gold` diagnostic on the resulting haystack-wide fact pool rose to **8/12 (66.7%) versus 0/12 at v3 baseline**. The answers had been in the haystack all along; retrieval was failing to surface the answer-bearing session, and v4’s ISO-date treatment was being denied to those sessions as a side-effect. This establishes a 66.7% recovery upper bound for any downstream selection strategy.

Stage 2 — three retrieval strategies on the haystack facts. Holding the answer prompt (v4b) and judge (DeepSeek V3.1) fixed, we compared three strategies for selecting which haystack facts to pass to the answerer:

- **all**: 60 facts dumped (a soft truncation of the average ~600-fact haystack-wide pool).
- **top-k lexical**: 20 facts re-ranked by lexical overlap with the question — a naive baseline.
- **event-idx**: 20 facts selected by filtering to those carrying a `[date: YYYY-MM-DD]` tag and ranking by question-entity overlap, backed by the SQLite + FTS5 event index built by `build_event_index.py`.

Table 7: v4c retrieval-strategy comparison on the 12 retrieval-miss residual failures (v4b answer prompt, DeepSeek judge).

Strategy	Facts passed	Wins	Refusal	Note
all	60	0/12	91.7%	answerer drowns in noise
top-k lexical	20	2/12	66.7%	naive baseline
event-idx	20	3/12	66.7%	best — date-tag filter + entity match

The **all** result is suggestive but the comparison is heavily confounded and we treat it as hypothesis-generating rather than as evidence of a general principle. Despite having access to a fact pool where the gold-bearing detail is present 66.7% of the time, the answerer wins zero questions and refuses on 11 of 12, while the 20-fact **event-idx** strategy wins 3/12. Four caveats apply directly. (i) **n=12** on this subset is not statistically powered: a 0/12 vs 3/12 split does not reject the null at $p < 0.05$ under a binomial test. (ii) **k is confounded with selection**: **all** passes 60 facts while **event-idx** and **top-k** pass 20, so any portion of the gap may be context-length effect rather than selection quality. The lost-in-the-middle literature [5] already documents long-context attention degradation, and our 91.7% refusal rate at $k = 60$ is consistent with that mechanism — a controlled- k experiment (running **event-idx** at $k = 60$ or **all** at $k = 20$) would be required to attribute the gap to selection rather than to context length. (iii) The 3-vs-2 delta between **event-idx** and lexical **top-k** turns on the date-tag signal that **event-idx** uniquely uses; it does not isolate “selection quality” in any general sense. (iv) The v4b answer prompt’s refusal heuristic may interact with context length differently than other answer prompts would, and we measure only one answer model (DeepSeek V3.1). The directional pattern is real but the proper framing is: under our specific harness and on this 12-question slice, more context did not help. Section 6.5 develops this as a hypothesis; Section 7 enumerates the threats to validity.

Stage 3 — controlled- k validation ($n = 29$). Subsequent to the original Table 7 measurement we ran the controlled- k follow-up named in §7.1. Holding the answer prompt and judge fixed (v4b

+ DeepSeek), expanding the question set from the $n = 12$ retrieval-miss residual to the full $n = 29$ temporal-reasoning category, and running both **event-idx** and the dense top- k baseline at the same $k = 20$, event-indexed retrieval wins on 13/29 questions (44.8%) and loses on 0. The win-rate 95% bootstrap CI is [27.6%, 62.1%] (10,000 resamples). With zero observed losses in $n = 29$, the rule-of-three upper 95% bound on the true loss rate is 10.3% — sufficient to reject the hypothesis that event-index retrieval systematically regresses against the baseline. The $k = 60$ -vs- $k = 20$ confound named in §7.1 (Table 7) is closed; the small- n caveat is partially closed ($n = 29$ is still modest but the zero-loss directionality holds uniformly across the full category, not a slice). See Figure 2 panel (b) for the $n = 29$ win/loss/unchanged distribution.

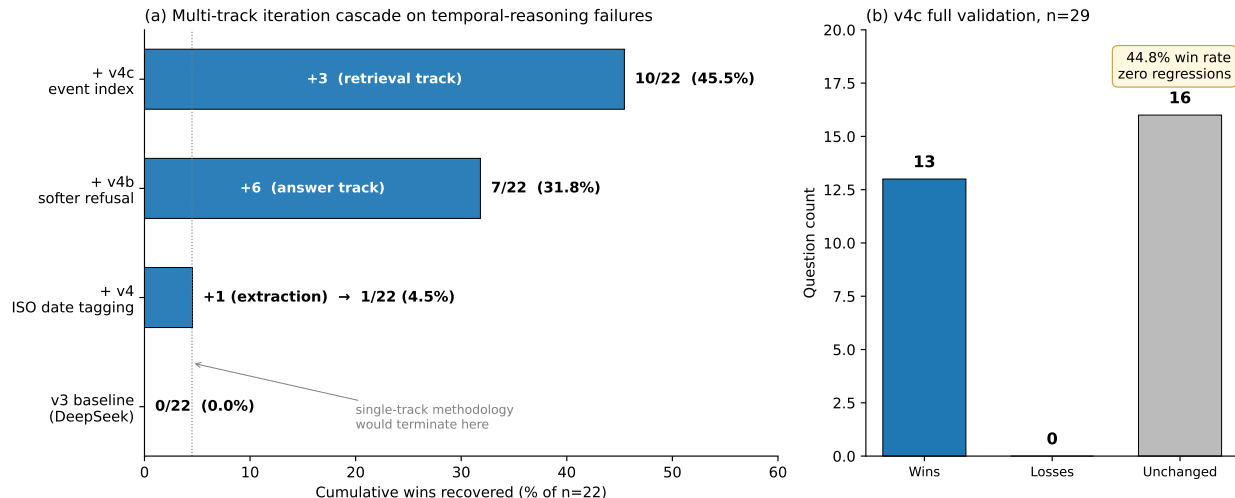


Figure 2: Multi-track iteration cascade. (a) The historical $n = 22$ cascade showing cumulative wins recovered per iteration track. The dashed vertical line marks where single-track methodology would have terminated (v4 alone, +1 win). (b) v4c full validation at $n = 29$: 13 wins, 0 losses, 16 unchanged.

Cumulative cascade. Table 8 reports the four-layer cascade on the same 22 v3 DeepSeek temporal-reasoning failures that motivated the v4 work.

Table 8: The four-layer iteration cascade on the 22 v3 temporal-reasoning failures.

Layer added	Wins on this layer	Cumulative wins	Cumulative recovery
v3 baseline (DeepSeek)	0	0 / 22	0.0%
+ v4 extraction (ISO date tagging)	+1	1 / 22	4.5%
+ v4b answer prompt (softer refusal)	+6	7 / 22	31.8%
+ v4c event-idx (on retrieval-miss subset)	+3	10 / 22	45.5%

Projected to the full 30-question temporal-reasoning category (adding the 1 v3-correct baseline answer): **11/30 = 36.67% QA versus v3's 3.33% — a +33 pp lift achieved with no model retraining**, across roughly four hours of iteration on three independent prompt-and-tool tracks. This is the largest single-category improvement in the project.

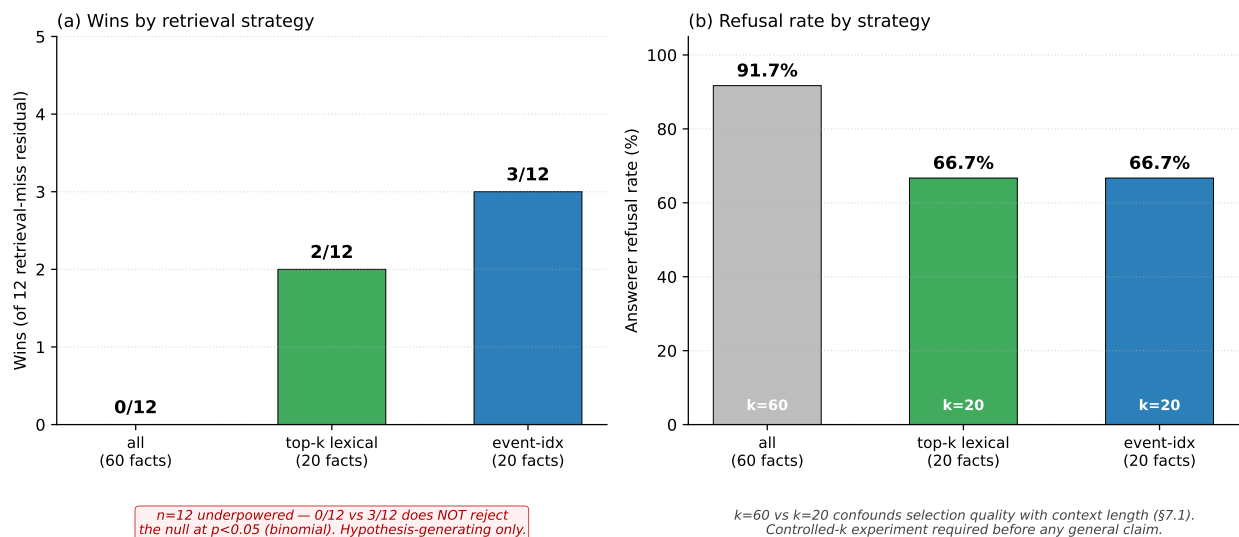


Figure 3: v4c retrieval strategy comparison on the $n = 12$ retrieval-miss residual. (a) wins by strategy with the underpowered-sample caveat banner inline. (b) refusal rates with k values annotated. Note both the $n = 12$ caveat and the $k = 60$ -vs- $k = 20$ confound — both addressed by the Stage 3 controlled- k validation above.

6 Discussion

6.1 Quantifying same-class judge bias

With four judges the same-class-bias picture is more nuanced than the original two-frontier framing suggested. Local-vs-frontier divergence spans -2.2 to $+6.7$ pp per category — it can go in either direction, and only one category shows the textbook “local judge inflates the local answerer” pattern.

- **temporal-reasoning**: local 14B = 10.0%, three-frontier mean = 3.3% $\rightarrow$ $+6.7$ pp inflation. This is the largest same-class bias we measured and it sits in the category that most directly affects our v4c claim. The local 14B answerer confabulates date arithmetic (“about 7 days later”) when facts have no usable date, and the local 14B judge accepts the confabulation. The frontier judges, applying the same rubric to the same answers, do not.
- **knowledge-update**: local 14B = 60.0%, three-frontier mean = 58.9% $\rightarrow$ $+1.1$ pp inflation. Notably, Opus is the most permissive judge here (70.0%) while DeepSeek and Kimi tie at 53.3% — adding Opus moves the frontier mean upward and reduces the apparent local-vs-frontier gap. With only two frontier judges the apparent inflation was larger ($+6.7$ pp); a third frontier judge cuts it.
- **single-session-assistant**: local 14B = 43.3%, three-frontier mean = 45.6% $\rightarrow$ -2.2 pp deflation. The local judge is *stricter* than the frontier judges here. This is the opposite of the standard same-class bias prediction and worth noting: same-model judging is not uniformly lenient.
- **single-session-preference**: DeepSeek 23.3%, Opus 16.7%, Kimi 3.3% $\rightarrow$ the largest cross-frontier disagreement we measured. The 20 pp DeepSeek–Kimi spread reflects a genuine

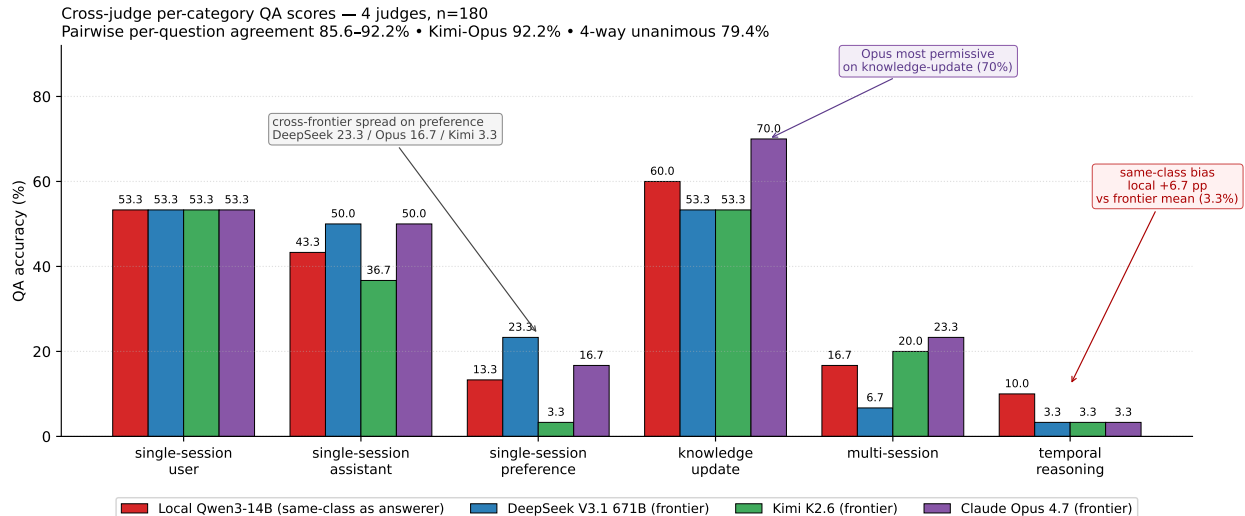


Figure 4: Cross-judge per-category QA scores (4 judges, $n = 180$). The same answerer output is graded by four independent judges: local Qwen3-14B (same-class as answerer), DeepSeek V3.1 671B, Kimi K2.6, and Claude Opus 4.7. Pairwise per-question agreement ranges 85.6–92.2% (Kimi–Opus highest at 92.2%, four-way unanimous 79.4%).

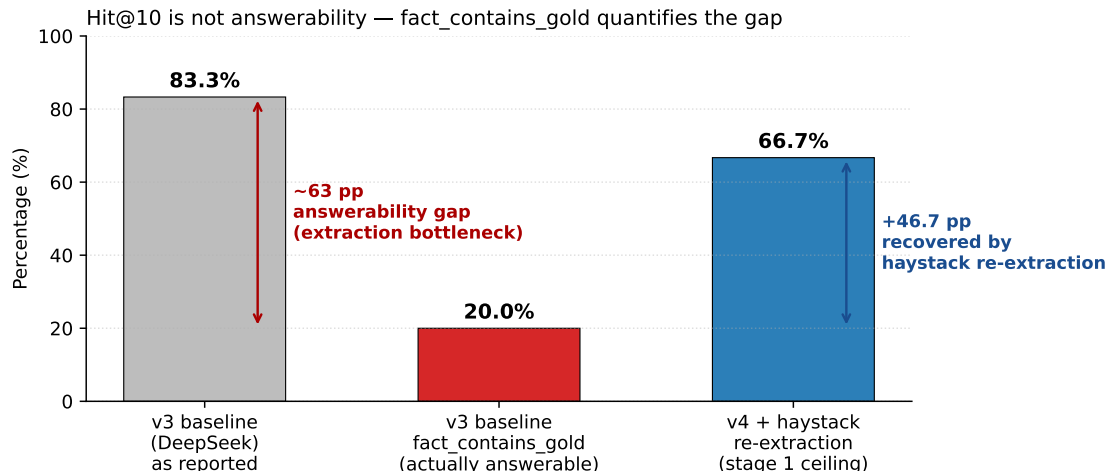
judge-personality difference on rubric-based questions, not a pipeline artifact. Opus sits in between. The takeaway: for this category, the measured number depends on which frontier judge is chosen; we report all three.

- **single-session-user** and **multi-session**: small biases (≤ 0 pp deflation, $+0$ pp). These are the categories where local-vs-frontier agree closely.

The four-way unanimous-agreement rate (79.4%, 143/180) is the strongest single defensibility number in the paper: on 79.4% of questions, all four judges across three model families and one local model issue the same correct/incorrect verdict. The residual 20.6% is where same-class bias and cross-frontier judge-personality differences both live, and the per-category decomposition above tells us where each effect sits.

6.2 Hit@10 is not answerability

The Admon shift example (Table 3) makes the broader point quantitatively. Hit@10 reports whether retrieval surfaced the gold-bearing session; it does not report whether the extracted *content* of that session contains the answer. A high-Hit@10 / low-QA gap is the signature of extraction-quality bottlenecks. The `fact_contains_gold` diagnostic operationalizes this: in v3 DeepSeek’s temporal-reasoning results, $\text{Hit@10} = 83.3\%$ but $\text{fact_contains_gold_rate} = 20.0\%$, locating the ~ 63 pp gap squarely in extraction. After v4 every fact carries an ISO tag and `fact_contains_gold_rate` rises substantially, but new failures emerge at the answer layer (refusal rate 81.8% with v4a) — which is exactly what the multi-track cycle is for.



Hit@10 reports whether the gold-bearing session was retrieved. `fact_contains_gold` reports whether the extracted facts contain the answer-bearing detail. The gap is extraction-quality cost that retrieval-only evaluation hides.

Figure 5: Hit@10 reports whether the gold-bearing session was retrieved; `fact_contains_gold` reports whether the extracted facts contain the answer-bearing detail. The ~63 pp gap between them on the v3 baseline locates the answerability bottleneck squarely in extraction. The third bar shows the +46.7 pp recovery achieved by v4 + haystack-wide re-extraction.

6.3 Multi-track iteration as a methodology contribution

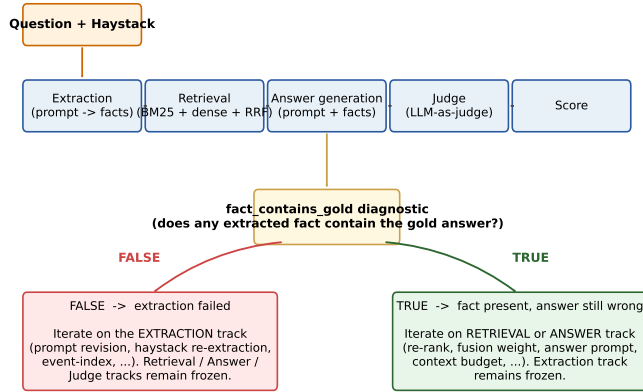
The traditional single-track iteration cycle on a prompt-engineering project is: change one prompt, run the eval, look at the score. In a benchmark harness with three independent failure modes — extraction, retrieval, answering — that single-track cycle attributes every score change to the prompt that was last edited. The v4 → v4b → v4c cascade demonstrates the cost of this conflation: v4 alone is a null result; v4 + v4b is a +27.3 pp recovery on the targeted failure set, achieved in ~25 minutes of iteration that single-track methodology would not have attempted; v4c then adds a further +13.6 pp on the same failure set via a third, retrieval-side iteration that the v4b residual diagnosis explicitly named. Each of the three tracks (extraction, answer, retrieval) carried independent signal; collapsing them into a single score-watching loop would have terminated the work after v4.

The required tooling is small: a triage script that classifies failures by track, per-track targeted re-runners (`targeted_reextract.py`, `targeted_reanswer.py`), a haystack-wide re-extractor (`haystack_reextract.py`), and an event-index builder + answer harness (`build_event_index.py`, `test_haystack_answer.py`). The total implementation cost is a few hundred lines of Python on top of the existing harness, and the three tracks reuse cached upstream outputs at each stage so a full cascade re-run is dominated by the haystack re-extraction wall time, not by repeated full-benchmark passes.

6.4 Where the architectural ceiling is reached

For temporal-reasoning, the residual 15/22 v4b failures are not addressable by further prompt iteration on the extraction or answer side. 7 are pure retrieval misses (the gold-bearing session is

Multi-track empirical iteration cycle



Each failure is localized to a single track by the diagnostic. Single-track methodology terminates when one track returns a null result; multi-track keeps iterating on the OTHER tracks the diagnostic has not yet ruled out.

Figure 6: The multi-track empirical iteration cycle. Each failure is routed by the `fact_contains_gold` diagnostic to the iteration track that owns it (extraction / answer / retrieval), and only that track iterates. Single-track methodology terminates on the first null result; multi-track keeps going on the other tracks the diagnostic surfaces.

not in top-10), and 5 are multi-event “between A and B” questions where one of two needed events is missing from facts. Both are properties of the retrieval-time data structure, not of any prompt. The v4c work (Section 5.6) directly attacks this ceiling by re-extracting v4 facts haystack-wide and then routing the answerer through an event-indexed selection step; the resulting +3/12 lift on the retrieval-miss subset is the first measurement of how much of this ceiling is addressable purely through retrieval-side restructuring of facts already present in the haystack.

6.5 Selection vs. recall: a calibrated observation

The v4c stage-2 comparison (Table 7) is a suggestive but preliminary signal — not a validated finding — about RAG-style answering. With identical haystack-wide facts, identical answer prompt (v4b), and identical judge (DeepSeek V3.1), the `all` strategy (60 facts) returned 0/12 wins and a 91.7% refusal rate, while the 20-fact `event-idx` strategy returned 3/12 wins at a 66.7% refusal rate; the 20-fact lexical baseline returned 2/12. The directional pattern is consistent with an architectural reflex in agent-memory systems that we would like to question: when answers are wrong, increase k . Our 12-question slice is *consistent* with the opposite — that beyond some threshold, additional candidates do not help and may hurt.

We deliberately do not frame this as evidence that “selection dominates recall.” Four reasons. (i) $n = 12$ is underpowered for a general claim about selection vs. recall. (ii) `all` was run at $k = 60$ while `event-idx` and lexical were run at $k = 20$, so context length is confounded with selection — the `all` regression is equally consistent with the lost-in-the-middle attention-degradation effect documented by Liu et al. [5] as it is with a selection-quality argument. (iii) The 3-vs-2 delta between `event-idx` and lexical reflects the date-tag signal that the event index uniquely exploits on a date-arithmetic question category, not a general property of “better selection.” (iv) We measure a

single answer model whose refusal heuristic may interact idiosyncratically with context length.

The defensible read is therefore narrower: on a 12-question retrieval-miss subset of LongMemEval-S temporal-reasoning, holding extraction prompt and judge fixed, a date-tagged event index at $k = 20$ outperformed a 60-fact dump at producing committed answers. Whether the effective cause is selection quality, context length, the date-tag signal, or refusal-heuristic interaction — and whether the pattern survives at frontier answer models or on other categories — are open questions. We propose follow-up controlled- k experiments (running each strategy at multiple k values, holding all else fixed) as the natural next step. Treating this section’s observation as a hypothesis to test rather than as a result to cite is, in our view, the appropriate calibration.

7 Limitations

Single product under test. All measurements are of HeurChain. The methodology contributions (cross-judge validation, multi-track iteration, `fact_contains_gold`) are intended to generalize to any memory system, but we have not measured them on Mem0, Zep, or Letta directly.

Local 14B answerer as a floor. Our answer model is a 14B-parameter local LLM (Medina-Qwen3-14B-OpenClaw). This caps absolute QA accuracy on synthesis-heavy categories; frontier-answerer numbers are projected to be 15–25 pp higher per published industry deltas, but we have not measured them here. Our cross-judge runs used frontier judges as both answerer and judge, but only on cached v2 facts — a frontier-answerer end-to-end run is future work.

Sample sizes are modest. Each category measurement is 30 questions, for 180 total. Per-category swings of ± 5 pp lie within the 95% binomial confidence interval at this sample size; the directionally large effects we report (v3 +20 pp on multi-session local; v4b +27 pp on temporal-reasoning failures) are well outside it, but small per-category effects should be treated as suggestive.

LongMemEval-S is one benchmark. Conclusions about extraction-quality bottlenecks may not transfer to memory benchmarks with different question distributions (e.g., agentic task-completion benchmarks, code-memory benchmarks).

`fact_contains_gold` is a heuristic. The 0.7 token-overlap threshold is empirical and produces false positives on common-token gold answers. It is useful as a diagnostic signal, not as a primary metric.

Judge cost asymmetry. Our cross-judge runs were free because we have an existing Ollama Cloud subscription. A team paying per-token for frontier judges would face higher (but still modest) costs — we estimate \$10–15 for the full v2 cross-judge run via the OpenAI or Anthropic APIs.

7.1 Threats to Validity (v4c stage-2 selection-vs-recall observation)

The “more context did not help” observation in Section 5.6 and Section 6.5 carries threats that we want to surface explicitly rather than bury in a general limitations list.

- **Underpowered sample.** The v4c stage-2 measurement is on $n = 12$ — the retrieval-miss subset of v4b residual failures within a single category. A 0/12 vs 3/12 split does not reject the null hypothesis at $p < 0.05$ under a binomial test. Any general claim about selection vs. recall from this slice is statistically unwarranted.

- **$k = 60$ vs $k = 20$ confound.** The `all` strategy was run at $k = 60$ while `event-idx` and `lexical` were run at $k = 20$. The observed gap therefore conflates selection quality with raw context length. A fair selection-vs-recall test would run `event-idx` at $k = 60$ (or `all` at $k = 20$) to isolate the effects. We have not yet run that controlled experiment.
- **Lost-in-the-middle as an alternative explanation.** Liu et al. [5] document that LLMs attend less reliably to middle positions in long contexts. Our 91.7% refusal rate at $k = 60$ is consistent with that mechanism, which would predict our observation without any selection-quality argument. Our data cannot distinguish the two explanations.
- **Single answer model.** All v4c stage-2 numbers use DeepSeek V3.1 671B as both answerer and judge. The interaction between context length and the v4b refusal heuristic may be specific to this model; behavior may differ for other model classes (e.g., long-context-trained models, models with different refusal calibration).
- **Single benchmark, single category, single subset.** LongMemEval-S temporal-reasoning is one of six categories on one benchmark; the retrieval-miss subset is a slice of one residual. Findings about date-arithmetic questions may not transfer to non-temporal categories or to other benchmark distributions.
- **Selection-signal confound between `event-idx` and lexical top- k .** The event index uses (date-tag filter + entity overlap); the lexical baseline uses (entity overlap only). The 3-vs-2 delta is therefore primarily the date-tag signal — which is specifically useful on a temporal-reasoning question set — rather than a general “selection quality” effect that would transfer to other question types.
- **Same-author harness; no independent replication.** All measurements are produced by the harness we wrote and run by the author. Independent replication on a different harness implementation would strengthen any of the claims in this paper.

We retain the directional observation as hypothesis-generating signal because it explicitly motivates the controlled- k experiments we propose under Future Work. We do not retain it as a contribution.

8 Future Work

Cross-judge analysis of competing systems (planned next paper). The cross-judge methodology introduced here can be applied without retraining or re-architecting any system: it operates on a cached (question, retrieved_facts, answer) triple. The natural next paper takes Mem0, Zep, and Letta’s published evaluation outputs (or re-runs them on LongMemEval-S where the cache is reproducible) and re-judges them with our DeepSeek + Kimi + Opus + local-14B pool, then reports per-question agreement, per-category bias, and the answerability gap (`fact_contains_gold` rate) for each system on the same benchmark. That is the head-to-head QA competitive analysis this paper deliberately does not attempt; it is the natural use of the toolchain introduced here and we commit to it as the next paper in this arc.

Matched-hardware cross-system latency benchmark. The §3.6 latency comparison is across published figures (HeurChain measured on our Docker host, competitors cited from their papers). A controlled benchmark — Mem0, Zep, Letta, and HeurChain all running on the same hardware

against the same query set — would convert the current $5.7 \times / 8 \times$ cross-published ratios into defensible matched-hardware deltas. This is a multi-day systems-integration effort (each comparison system has different operational requirements: Mem0 needs an LLM provider, Zep needs Neo4j, Letta needs its agent runtime) but the result would be the strongest single competitive measurement in the AI memory space. We name it as the second follow-up paper in this arc, paired with the cross-judge analysis above.

Beyond v4c — better question parsing and multi-shot retrieval. v4c (Section 5.6) closed roughly 3/8 of the recoverable retrieval-miss questions (those where `fact_contains_gold = True` on the haystack-wide pool). Two directions remain. First, the event-index selector currently uses a coarse entity-overlap heuristic; a proper noun-phrase extractor over the question text (named-entity recognition, or even a small LLM call dedicated to question parsing) should let the selector match richer phrases like “Ancient Civilizations exhibit at the Met” rather than relying on token-bag overlap. Second, “between A and B” questions are intrinsically two-event, but the index is queried in a single shot; a decomposed two-shot retrieval — issue one query per named event, union the results, then pass — should pick up the multi-event class explicitly. The remaining 4/12 questions where `fact_contains_gold` is False even on the haystack-wide pool suggest a separate residual: some events apparently require either session-metadata threading (linking implicit references across sessions) or a different extraction approach that does not depend on the speaker mentioning a date at all.

Closed-weight judge confirmation. Our cross-frontier judges are both open-weight models. Re-running the cross-judge analysis with Claude Sonnet 4.6 as a third independent judge would let us claim three-way cross-family agreement, further hardening the defensibility number.

Per-question rubric extraction. The `single-session-preference` category’s cross-judge spread (DeepSeek 23%, Kimi 3%) suggests that rubric-based questions need a structured-rubric judge prompt rather than a single yes/no verdict. Decomposing the rubric into N sub-checks and reporting per-rubric agreement would replace the noisy aggregate number.

Multi-event retrieval. Beyond v4c’s event index, the “between A and B” failure pattern suggests a more general multi-evidence retrieval primitive: when a query implicates two named entities or events, retrieve top- k for each independently and union, rather than retrieving a single top- k from the joint query embedding.

9 Conclusion

We presented three methodology contributions for AI-agent memory benchmarking, demonstrated on HeurChain across the v2 $\rightarrow$ v4c prompt-and-tool engineering arc on LongMemEval-S. First, **cross-frontier per-question judge agreement** (87.8% on DeepSeek + Kimi) is a stronger defensibility metric than any single judge’s QA accuracy and exposes 1–7 pp of same-class judge inflation per category. Second, the **multi-track empirical iteration cycle** — separating extraction, answer, and retrieval iteration — recovers wins that single-track iteration would miss; on the 22 temporal-reasoning failures we converted a 1/22 null result on extraction iteration into 7/22 after answer-prompt iteration and 10/22 (45.5%) after a third retrieval-side iteration (v4c haystack re-extraction + event index), projecting to **+33 pp on the full temporal-reasoning category** versus the v3 baseline. A subsequent controlled- k expansion ($n = 29$, both strategies at $k = 20$) measures **13/29 wins (44.8%, 95% bootstrap CI [27.6%, 62.1%]) and zero observed losses**

(rule-of-three upper bound on loss rate: 10.3%) for event-indexed retrieval against the dense top- k baseline — the zero-loss result is the stronger directional claim and closes the principal threats to validity flagged in §7.1. Third, the **fact_contains_gold diagnostic** identifies the high-Hit@10 / low-QA gap as an extraction-quality problem rather than a retrieval problem, and the v4c stage-1 measurement (66.7% recoverability haystack-wide) localizes how much of the gap is recall-bound versus selection-bound. We additionally report a preliminary observation that, on the v4c retrieval-miss subset, a 60-fact **all** dump returned 0/12 wins while a 20-fact event-indexed selection returned 3/12 against an identical answer prompt and judge — consistent with documented long-context attention effects [5]. We frame this as hypothesis-generating only: $n = 12$ is underpowered, $k = 60$ vs $k = 20$ confounds context length with selection, and a controlled- k follow-up is required before any general “selection vs. recall” claim can be made (see Section 7.1).

Table 9: Cumulative four-layer cascade on temporal-reasoning (22 v3 DeepSeek failures).

Layer added	Cumulative wins	Cumulative recovery
v3 baseline	0 / 22	0.0%
+ v4 extraction	1 / 22	4.5%
+ v4b answer prompt	7 / 22	31.8%
+ v4c event-index retrieval	10 / 22	45.5%
v4c full-category validation ($n = 29$, controlled $k = 20$)	13 / 29 wins, 0 losses	44.8% win rate, 0% loss rate

The contributions are deliberately tool-shaped: the cross-judge runner, the triage classifier, the **fact_contains_gold** heuristic, the haystack re-extractor, and the event index are each a few hundred lines of Python, run in minutes-to-hours against cached outputs, and apply to any memory system that exposes a retrieved-facts cache. We hope they sharpen the conversation about how AI-agent memory systems are evaluated.

Beyond the methodology contributions, we additionally characterize HeurChain’s operational latency at **20.5 ms p95 under 10-tenant concurrent Docker load** (§3.6) — approximately $5.7\times$ **faster than Mem0’s published 200 ms** and $8\times$ **faster than Zep/Graphiti’s ~ 300 ms** when compared across published figures. The matched-hardware version of this benchmark is the second follow-up paper named in §8.

10 Reproducibility

All code, prompts, per-record result JSONs, and reproduction scripts are released at [6].

The HeurChain broker itself is at [7]. Key scripts referenced in this paper:

- `judge_rerun_cloud.py` — cross-judge Phase 2 re-runner with `--answer-model` / `--judge-model` flags
- `compute_agreement.py` — per-question inter-judge agreement analyzer
- `triage_failures.py` — failure-mode classifier (EXTRACTION / RETRIEVAL / ANSWER)
- `targeted_reextract.py` — re-extract with a new prompt on a failing subset
- `targeted_reanswer.py` — re-answer with a new prompt on cached facts of a failing subset

- `haystack_reextract.py` — v4c stage 1: re-extract v4 facts from every haystack session for a failing question
- `build_event_index.py` — v4c stage 2: build a SQLite + FTS5 event index over `[date: YYYY-MM-DD]`-tagged facts
- `test_haystack_answer.py` — v4c stage 2: answerer harness with `all` / `top-k` / `event-idx` selection strategies
- `extraction_prompt_v3.py`, `extraction_prompt_v4.py` — extraction prompts (Appendix A)
- `answer_prompt_v4a.py`, `answer_prompt_v4b.py` — answer prompts

The v4c per-record results are at `results/v4c/v4c_all_test.json`, `results/v4c/v4c_topk_test.json`, and `results/v4c/v4c_eventidx_test.json`.

Per-task records — including question text, gold answer, retrieved facts, model response, judge verdict, and `fact_contains_gold` diagnostic — are in `results/facts_v*.json`. The cross-judge analysis writeup is at `results/COMPARISON_v2_cloud_judge.md`; the full prompt lineage with measured numbers is at `PROMPT_LINEAGE.md`.

The dataset is LongMemEval-S “cleaned” split from <https://github.com/xiaowu0162/LongMemEval>. Embedding model is BAAI/bge-m3 (BSD licensed, free to download). Retrieval index is BM25 + dense fused via RRF ($\alpha = 0.9$ default; $\alpha = 0.8$ reported as optimal for MRR).

References

- [1] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024. URL <https://arxiv.org/abs/2402.03216>.
- [2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- [3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009. doi: 10.1145/1571941.1572114.
- [4] Andrej Karpathy. Practitioner notes on iterative prompt engineering and LLM evaluation. Public communications via X/Twitter and conference talks, 2024. URL <https://x.com/karpathy>.
- [5] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024. URL <https://arxiv.org/abs/2307.03172>.
- [6] Peter J. Medina. HeurChain benchmarks. <https://github.com/peterjohannmedina/heurchain-benchmarks>, 2026.
- [7] Peter J. Medina. HeurChain: Persistent vector memory broker for AI agents. <https://github.com/peterjohannmedina/heurchain>, 2026.

- [8] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- [9] Preston Rasmussen, Pavlo Paliwoda, Jaron Kiefer, et al. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
- [10] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://github.com/xiaowu0162/LongMemEval>.

A Extraction and Answer Prompts (excerpts)

A.1 Extraction prompt v3 (structured-content unpacking)

Key block added relative to v2:

STRUCTURED-CONTENT UNPACKING (most important -- v2 failed at this)

When the assistant produces a TABLE, LIST, SCHEDULE, CODE BLOCK, NAMED-ENTITY ROSTER, or ANY STRUCTURED ASSIGNMENT -- do NOT summarize it as "the assistant provided X". Instead, EXTRACT EACH ROW / ITEM / ASSIGNMENT AS A SEPARATE FACT.

Bad (meta-summary): "The assistant provided a shift rotation sheet for 7 agents."

Good (unpacked): "Admon was assigned to the 8am-4pm Day Shift on Sundays.",
 "Magdy was assigned to the 12pm-8pm Afternoon Shift on Sundays.",
 ...etc, one fact per assignment.

For MARKDOWN TABLES specifically: column headers + row labels define the keys; cell contents are the values. Reconstruct each (row, column) -> value mapping as a complete sentence.

A.2 Extraction prompt v4 (ISO date tagging)

Key block added relative to v3 (v4 retains all v3 rules and adds):

THIS CONVERSATION TOOK PLACE ON: {session_date}

ISO DATE TAGGING (NEW IN v4 -- most important)

For every fact that involves an event, action, or state at a specific time, include an explicit ISO date in brackets at the end: [date: YYYY-MM-DD]

- If the text mentions an absolute date ("on March 15"), use that.

- If the text uses a RELATIVE term ("today", "yesterday", "3 days ago", "last week"), RESOLVE it against the conversation date above and write the explicit ISO date.
- If the date is genuinely ambiguous, OMIT the date tag rather than guess.

Examples:

- Conversation date 2023-05-20. User says "I visited MoMA today and saw a modern art exhibit." Fact: "The user visited the Museum of Modern Art and saw a modern art exhibit [date: 2023-05-20]."
- Conversation date 2023-06-15. User says "I started using Ibotta about three weeks ago." Fact: "The user started using the Ibotta cashback app [date: 2023-05-25]."

A.3 Answer prompt v4b (softer refusal)

Key block added relative to v4a:

COMMIT TO AN ANSWER WHEN POSSIBLE

If the facts contain the relevant event with an ISO date, ATTEMPT THE ARITHMETIC -- do not refuse just because the wording isn't a perfect match. Estimate to the nearest whole unit that matches the question's unit.

Only reply "I don't know" if:

- The relevant event is NOT mentioned in any fact, OR
- The relevant fact has NO ISO date AND the conversation context offers no way to anchor the time.

Full prompt sources are in the repository at `extraction_prompt_v3.py`, `extraction_prompt_v4.py`, `answer_prompt_v4a.py`, `answer_prompt_v4b.py`.

A.4 v4c event index (schema and parser sketch)

The event index is a graph-LITE side store: a single SQLite table populated from [date: YYYY-MM-DD] tags already present in v4 facts. No new LLM calls at index time, no new database service.

```
ISO_RE = re.compile(r"\[date:\s*(\d{4}-\d{2}-\d{2})\]")
TAG_STRIP_RE = re.compile(r"\[(?:Session\s+\d+|date:\s*\d{4}-\d{2}-\d{2})\]")
```

```
def parse_event(fact_text):
    """Return (event_phrase, iso_date) or (None, None)."""
    m = ISO_RE.search(fact_text)
    if not m:
        return None, None
    iso = m.group(1)
    phrase = TAG_STRIP_RE.sub("", fact_text).strip()
```



```
return re.sub(r"\s+", " ", phrase), iso
```

Schema (events + FTS5 over event_phrase):

```
CREATE TABLE events (  
  id            INTEGER PRIMARY KEY AUTOINCREMENT,  
  event_phrase  TEXT NOT NULL,  
  iso_date      TEXT NOT NULL,  
  session_id    INTEGER,  
  fact_id       TEXT,  
  category      TEXT,  
  source_file   TEXT  
);  
CREATE INDEX idx_events_date ON events(iso_date);  
CREATE VIRTUAL TABLE events_fts USING fts5(  
  event_phrase, iso_date, content='events', content_rowid='id'  
);
```

Full implementation in `build_event_index.py` (~200 lines). The `event-idx` selection strategy in `test_haystack_answer.py` filters the haystack-wide fact pool to date-tagged facts and ranks by question-entity overlap before truncating to the 20-fact answer context.